Taller práctico 4 - Colas de prioridad y árboles binarios de búsqueda

Curso: Estructuras de Datos y Algoritmos

Período: 2026-20

Facultad: TIC - UPB

Modalidad: Equipos de máximo 3 personas

Objetivos

Integrar en un mismo problema las dos estructuras de datos vistas recientemente: la **tabla de símbolos ordenada** (implementada con arreglos ordenados y búsqueda binaria, con **árboles binarios de búsqueda** y con **árboles rojo-negros**) y la **cola de prioridad** (implementada con arreglo desordenado, arreglo ordenado y **montículo binario**). El problema propuesto, un índice de frecuencias de palabras con consultas de *top-K* y de autocompletado, requiere ambas estructuras de forma esencial. Además permite observar experimentalmente tres fenómenos: la dependencia del costo de un BST respecto de la **forma del árbol** (y por tanto del **orden de inserción**), el efecto de un parámetro de la consulta (K) sobre el costo de la cola de prioridad, y el hecho de que el tamaño de la tabla de símbolos (M) **no** crece linealmente con el tamaño del texto (N).

Un objetivo transversal es aprender a **expresar el costo en función del parámetro correcto**: las operaciones de la tabla de símbolos dependen de M, las de la cola de prioridad de K, y el usuario solo controla N. Para traducir los resultados a N hace falta una relación empírica entre M y N que debe medirse.

> **Uso de IA en este taller:** En este período se **permite y se fomenta** el uso de asistentes de IA (ChatGPT, Claude, Copilot, etc.) en la parte de codificación, **siempre** siguiendo la metodología descrita en la **Guía para el uso efectivo de IA en el curso**. La idea central de la guía es: *la IA implementa; el estudiante decide, especifica y verifica.* En este taller la distinción es especialmente crítica: **el análisis analítico del punto 7 y la interpretación de los resultados experimentales de los puntos 6 y 8 son trabajo propio del equipo y no deben delegarse.** El uso de IA **sin** la documentación exigida (PRD, prompts, validación crítica y reflexión) se considera entrega incompleta. Ver la sección **Uso de asistentes de IA** y el entregable `desarrollo-ia.md`.

Notación

Para evitar confusiones entre los distintos tamaños que aparecen en este taller, se usará en todo el documento la siguiente notación:

- N: número de palabras (*tokens*) del texto procesado, con repeticiones.
- M: número de palabras **distintas** (tamaño del vocabulario = tamaño de la tabla de símbolos), $M \leq N$.
- K: número de palabras solicitadas en la consulta *top-K* (tamaño máximo de la cola de prioridad), $1 \leq K \leq M$.
- h: altura del árbol (número de enlaces en el camino más largo de la raíz a una hoja; un árbol de un solo nodo tiene altura 0).
- $\bar{d}$: profundidad promedio de los nodos del árbol (la raíz tiene profundidad 0). Una búsqueda exitosa de una llave a profundidad d realiza $d + 1$ comparaciones.
- $\bar{d}_f$: profundidad promedio **ponderada por frecuencia**, $\bar{d}_f = \frac{1}{N} \sum_w f(w)\, d(w)$, donde $f(w)$ es la frecuencia de la palabra w en el texto.
- Q: número de consultas de autocompletado; R: número de palabras del vocabulario que comienzan con un prefijo dado; $K' = 5$: número de sugerencias por consulta de autocompletado.

Contexto del problema

Se quiere construir el **índice de vocabulario** de un texto largo para soportar tres tipos de consulta:

1. **Frecuencia:** ¿cuántas veces aparece la palabra w?
2. ***Top-K*:** ¿cuáles son las K palabras más frecuentes del texto, en orden de relevancia?
3. **Autocompletado:** dado un prefijo p (por ejemplo `dul`), ¿cuáles son las K' palabras más frecuentes que comienzan por p? (respuesta esperada: `dulcinea`, `dulce`, `dulces`, ...).

La consulta 1 la resuelve una **tabla de símbolos** (palabra → frecuencia). La consulta 2 la resuelve una **cola de prioridad** acotada a K elementos. La consulta 3 requiere **ambas**: la tabla de símbolos debe ser **ordenada**, porque las palabras que comienzan por p forman un **rango contiguo** de llaves, y la cola de prioridad selecciona entre ellas las K' más frecuentes.

El texto

Se usará el texto completo de *Don Quijote de la Mancha* de Miguel de Cervantes, disponible en Project Gutenberg (dominio público):

- Página: https://www.gutenberg.org/ebooks/2000
- Texto plano UTF-8: https://www.gutenberg.org/cache/epub/2000/pg2000.txt

Descargue el archivo **una sola vez** y use siempre la misma copia local. Todos los equipos deben producir exactamente los mismos conteos, así que la tokenización es parte del enunciado y **no** es opcional:

1. Descartar el encabezado y el pie de página de Project Gutenberg: usar solo el texto comprendido entre la línea que contiene `*** START OF` y la línea que contiene `*** END OF` (excluyendo ambas).
2. Convertir el texto a minúsculas.
3. Extraer las palabras como las secuencias maximales de caracteres que coinciden con la expresión regular `[a-záéíóúüñ]+`. Todo otro carácter (espacios, signos de puntuación, dígitos, comillas, guiones) es separador.

Con la copia del archivo disponible en octubre de 2026, esta tokenización produce $N = 383\,637$ palabras y un vocabulario de $M = 22\,944$ palabras distintas. Úselos como **valores de verificación**: si su programa obtiene otros valores, revise la tokenización antes de seguir. Si el archivo de Project Gutenberg cambia, las cifras pueden diferir ligeramente; en ese caso reporte las suyas e indique la fecha de descarga.

Orden de las llaves

Las llaves se comparan como cadenas en **orden lexicográfico por código de carácter** (orden *ordinal*), no en orden alfabético del idioma español. En este orden las letras con tilde y la `ñ` quedan **después** de la `z` (por ejemplo, la última llave del vocabulario es `útiles`, no una palabra que empiece por `z`). Esto es correcto y es lo que se espera:

- **Python:** la comparación nativa de `str` (`<`, `==`) ya es ordinal.
- **Java:** `String.compareTo` es ordinal.
- **C#: no** usar `string.CompareTo` ni `string.Compare` sin argumentos, que dependen de la cultura. Usar `string.CompareOrdinal(a, b)` o `StringComparer.Ordinal`.

Rango de un prefijo

Las palabras que comienzan por el prefijo p son exactamente las llaves w con $p \le w < p + \sigma$, donde σ es cualquier carácter mayor que todos los caracteres que pueden aparecer en una palabra. Use $\sigma =$ `'␣'`. **No** use caracteres como `'~'` o `'z'`: son menores que `'á'`, `'ñ'` o `'ü'` y harían perder palabras del rango. Justifique en el informe por qué el rango es contiguo en una tabla de símbolos ordenada.

Relevancia y desempate

Una `Entrada` es un par $(w, f(w))$. Se dice que una entrada es **más relevante** que otra si tiene **mayor frecuencia** o, a igual frecuencia, si su palabra es **menor** en el orden de las llaves. Este criterio es un orden total sobre las entradas y hace que la respuesta del *top-K* sea única. El desempate **no es un detalle**: en este texto hay empates exactamente en la frontera de $K = 100$ y de $K = 1000$, de modo que sin regla de desempate dos implementaciones correctas producirían respuestas distintas.

Valores de verificación para el texto completo:

- *Top-10:* `que` (20 769), `de` (18 410), `y` (18 272), `la` (10 492), `a` (9 876), `en` (8 285), `el` (8 267), `no` (6 346), `los` (4 769), `se` (4 752).
- Autocompletado de `dul` ($R = 12$): `dulcinea` (286), `dulce` (26), `dulces` (5), `dulcísima` (4), `dulcineas` (2).
- Autocompletado de `san` ($R = 53$): `sancho` (2 174), `sansón` (77), `sangre` (65), `santa` (49), `san` (32).
- `rank("quijote")` $= 17\,881$; `min()` $=$ `a`; `max()` $=$ `útiles`.

Ejercicios a desarrollar

1. ADTs base y tokenización

- **Tokenizador:** función que recibe la ruta del archivo y devuelve el arreglo de las N palabras en el orden del texto, según las reglas anteriores. Esta fase es de entrada/salida y **no** se cronometra.
- **ADT `Entrada`:** inmutable, con `palabra`, `frecuencia` y `toString()`. Debe ser **comparable** (`Comparable<Entrada>` en Java, `IComparable<Entrada>` en C#, `__lt__` en Python) según el criterio de **relevancia**: una entrada es *mayor* que otra si es más relevante. Así, una cola de prioridad de mínimos (`MinPQ`) mantiene en la cima la entrada **menos** relevante.
- Pruebas unitarias: verificar la tokenización sobre una cadena pequeña con mayúsculas, tildes, `ñ`, `ü`, dígitos y puntuación (por ejemplo, `"¿Qué? Señor, 2 pingüinos..."` → `["qué", "señor", "pingüinos"]`), y el criterio de comparación de `Entrada`, incluido el caso de frecuencias iguales.

2. Tablas de símbolos ordenadas

API común de tabla de símbolos ordenada (texto guía, Sedgewick y Wayne): `put(key, val)`, `get(key)`, `contains(key)`, `size()`, `isEmpty()`, `min()`, `max()`, `floor(key)`, `ceiling(key)`, `rank(key)`, `select(k)`, `keys()` y `keys(lo, hi)`. Las llaves son cadenas y los valores, enteros (frecuencias). Las tres implementaciones deben ser intercambiables bajo esta API.

Versiones:

- **`BinarySearchST`:** dos arreglos paralelos ordenados (`keys[]`, `vals[]`) con búsqueda binaria (capítulo de búsqueda). Arreglos redimensionables (duplicar al llenarse).
- **`BST`:** árbol binario de búsqueda sin balancear, con el campo `size` en cada nodo para soportar `rank` y `select` en tiempo $O(h)$.
- **`RedBlackBST`:** árbol rojo-negro inclinado a la izquierda (LLRB), con `size` en cada nodo.

Requisitos adicionales:

- `keys(lo, hi)` debe recorrer **solo** la parte del árbol (o del arreglo) que puede contener llaves del rango, y debe poder entregar también el **valor** asociado a cada llave (por ejemplo, con un método `entries(lo, hi)` que produzca pares, o un iterador de pares), para no repetir una búsqueda por cada llave del rango.
- `BST` y `RedBlackBST` deben proveer `height()` y `depthStats()`, que devuelve $\bar{d}$ y $\bar{d}_f$ (para $\bar{d}_f$ el valor de cada nodo es la frecuencia).
- **No** se implementa `delete` (ni `deleteMin`/`deleteMax`) en ninguna de las tres tablas de símbolos: el problema no lo requiere, y así las tres implementaciones ofrecen exactamente la misma API.
- Cada implementación debe proveer un verificador de invariantes, usado **solo** en las pruebas: `isBST()` (orden simétrico), `isSizeConsistent()` (el `size` de cada nodo es $1+$ el de sus hijos), `isRankConsistent()` (`rank(select(i)) == i` para todo i), y para `RedBlackBST` además `is23()` (ningún nodo con dos enlaces rojos seguidos ni enlace rojo a la derecha) e `isBalanced()` (todos los caminos de la raíz a un enlace nulo tienen el mismo número de enlaces negros). En `BinarySearchST`, verificar que `keys[]` esté estrictamente ordenado.

Cada implementación debe estar **instrumentada** con los siguientes contadores:

- **`compares`:** número de comparaciones de llaves. Cada comparación de tres vías entre la llave buscada y una llave almacenada cuenta como **1**, aunque el lenguaje la exprese con dos operadores. En Python, escriba un único método `compare(a, b)` que devuelva un entero negativo, cero o positivo e incremente el contador una sola vez; no use `<` y luego `>` sobre las llaves dentro de la estructura.
- **`moves`** (solo `BinarySearchST`): número de posiciones corridas en `put`. Mover un par (llave, valor) una posición cuenta como **1**.
- **`rotations`** y **`flips`** (solo `RedBlackBST`): número de rotaciones (izquierda o derecha) y de cambios de color (`flipColors`).
- Proveer `resetCounters()`.

Pruebas unitarias: árbol (o tabla) vacío, un elemento, actualización del valor de una llave existente (el tamaño no cambia), `floor`/`ceiling` de llaves presentes, ausentes, menores que `min()` y mayores que `max()`; `rank`/`select` en los extremos; `keys(lo, hi)` con rango vacío, con rango de un solo elemento y con `lo > hi`. Verificar que las tres implementaciones producen **la misma secuencia** `keys()` y los mismos valores para la misma secuencia de `put`. Verificar a mano el contador `compares` en un caso pequeño: por ejemplo, insertar en un `BST` vacío las llaves `"m"`, `"c"`, `"t"`, `"a"`, `"e"` en ese orden debe requerir exactamente $0 + 1 + 1 + 2 + 2 = 6$ comparaciones.

3. Colas de prioridad

API común: `MinPQ` genérica sobre elementos comparables, con `insert(x)`, `min()`, `delMin()`, `size()` e `isEmpty()`. Las tres implementaciones deben ser intercambiables bajo esta API y genéricas, no especializadas en `Entrada`.

Versiones:

- **Arreglo desordenado:** `insert` en $O(1)$; `delMin` busca el mínimo, lo intercambia con el último y lo retira.
- **Arreglo ordenado:** mantenido en orden **decreciente**, de modo que el mínimo está al final y `delMin` es $O(1)$. `insert` coloca el nuevo elemento al final y lo desplaza hacia el frente mediante intercambios sucesivos, como en el ordenamiento por inserción.
- **Montículo binario:** representado en arreglo, con `swim` y `sink`.

El montículo binario debe proveer además:

- Un **constructor a partir de un arreglo** que construya el montículo en tiempo lineal mediante *heapify* de abajo hacia arriba (`sink` desde $\lfloor n/2 \rfloor$ hasta 1), sin insertar uno a uno.
- La posibilidad de funcionar como montículo de **máximos** (por ejemplo, recibiendo un comparador o una bandera de orientación), necesaria para el punto **6e**.
- Un verificador `isMinHeap()` (o `isMaxHeap()`), usado solo en las pruebas.

Cada implementación debe estar instrumentada con contadores de **comparaciones** (`less`: cada llamada al comparador) y de **intercambios** (`exch`: cada intercambio de dos elementos, contado como 1), igual que en el Taller 3. Proveer operación para reiniciar los contadores.

Pruebas unitarias: `delMin` sobre cola vacía (comportamiento definido y documentado), inserción de elementos repetidos, inserción en orden creciente y decreciente, y verificación de que n llamadas a `delMin` devuelven los n elementos en orden creciente. Verificar que el constructor por *heapify* produce un montículo válido. Verificar a mano los contadores en un caso pequeño (por ejemplo, el arreglo desordenado con 5 elementos requiere exactamente 4 comparaciones en el primer `delMin`).

4. Selección de la combinación a evaluar

Procedimiento determinista (el mismo del Taller 3):

1. Ordenar los ID institucionales de los miembros del equipo alfabéticamente (como cadenas, con ceros a la izquierda si aplica).
2. Concatenarlos sin separadores en una sola cadena.
3. Calcular su SHA-256 (salida en hexadecimal). Puede usar la herramienta en línea (Calcular SHA-256) o `python3 -c "import hashlib; print(hashlib.sha256(b'...').hexdigest())"`.
4. Interpretar el hexadecimal como un entero en base 16 y calcular su **módulo 9**. El residuo se calcula sobre el entero completo, no sobre la suma de los dígitos hexadecimales: `python3 -c "print(int('<hash>', 16) % 9)"`.

El número obtenido selecciona la combinación de estructuras sobre la que se hará el análisis detallado:

Valor	Tabla de símbolos	Cola de prioridad
0	BinarySearchST	Arreglo desordenado
1	BinarySearchST	Arreglo ordenado
2	BinarySearchST	Montículo binario
3	BST	Arreglo desordenado
4	BST	Arreglo ordenado
5	BST	Montículo binario
6	RedBlackBST	Arreglo desordenado
7	RedBlackBST	Arreglo ordenado
8	RedBlackBST	Montículo binario

> Ejemplo: con `id1='0001234'`, `id2='0005678'`, `id3='0009012'`, la cadena es `000123400056780009012`, su SHA-256 es `f29e3c71f1bd144593e36474e7504823362453c0fb69623092aa8ca7b2505593`, y `mod 9 = 3`, lo que asigna **BST + arreglo desordenado**.

Reportar en el informe los ID ordenados, la cadena concatenada, el hash obtenido y la combinación asignada. Se deben implementar las tres variantes de cada familia (puntos 2 y 3), pero el análisis detallado de los puntos 6 y 7 se hace sobre la combinación asignada,

salvo en los puntos **6c** y **6e**, que todos los equipos hacen igual.

5. Construcción del índice y consultas

Implementar el programa completo, usando la combinación asignada:

1. **Lectura y tokenización** (no se cronometra): obtener el arreglo de palabras del texto completo. Para un tamaño N dado, usar las **primeras** N palabras del texto.
2. **Fase A: construcción del índice.** Para cada una de las N palabras, en el orden del texto, actualizar su frecuencia en la tabla de símbolos como en el programa `FrequencyCounter` del texto guía: `v = get(w)`; si `v` es nulo, `put(w, 1)`; si no, `put(w, v + 1)`. Use exactamente este esquema (un `get` seguido de un `put`, es decir, dos búsquedas por palabra) para que los contadores de todos los equipos sean comparables.
3. **Fase B: *top-K*.** Recorrer `keys()` (en orden de llave) y, para cada palabra w, ejecutar:

   ```
   pq.insert(Entrada(w, f(w)))
   si pq.size() > K: pq.delMin()
   ```

   Al terminar, la cola contiene las K entradas más relevantes. Extraerlas con K llamadas a `delMin` e invertir el resultado para presentarlo de la más a la menos relevante. Use exactamente este esquema, sin comparar antes con `min()`: la optimización se discute en el punto **7b**.
4. **Fase C: autocompletado.** El conjunto de consultas es el de todos los **prefijos distintos de longitud 2** de las palabras del vocabulario que tienen al menos 2 letras (para el texto completo, $Q = 295$), procesados en orden. Para cada prefijo p, recorrer las entradas del rango [p, `p + '␣'`) con `keys(lo, hi)` y seleccionar las $K' = 5$ más relevantes con una cola de prioridad acotada, con el mismo esquema de la fase B.

El programa debe medir **por separado** el tiempo de las fases A, B y C. Excluir de las mediciones la lectura del archivo, la tokenización y toda operación de E/S (impresión, escritura de resultados, graficación). Sugerencias por lenguaje: `time.perf_counter()` en Python, `System.nanoTime()` en Java (con calentamiento previo de la JVM), `Stopwatch` en C#. Si el tiempo de una fase está por debajo de la resolución del reloj, aumentar las repeticiones y promediar.

El programa debe reportar también, para cada ejecución, los contadores de la tabla de símbolos y de la cola de prioridad **por fase** (reiniciarlos al comenzar cada fase), M, y para los árboles h, $\bar{d}$ y $\bar{d}_f$ al terminar la fase A.

6. Evaluación experimental

Trabajar con los tamaños $N \in \{6\,250,\ 12\,500,\ 25\,000,\ 50\,000,\ 100\,000,\ 200\,000\}$ (progresión geométrica de razón 2), tomando siempre las primeras N palabras del texto. El texto completo ($N = 383\,637$) se usa en los puntos 6c a 6f y puede agregarse a las gráficas, pero no rompe la razón constante de la serie de ajuste.

Protocolo de repeticiones: los datos son deterministas, así que los **contadores deben ser idénticos** en todas las repeticiones de un mismo caso (es una prueba de sanidad de la instrumentación: si cambian, hay un error). Los **tiempos** sí varían: realizar al menos 10 repeticiones por tamaño y reportar promedio y desviación estándar. En los experimentos que usan permutaciones aleatorias (punto 6d), usar las semillas 1 a 10 y reportarlas.

Para cada tamaño registrar, con $K = 10$:

- N, M y Q.
- Tiempo de cada fase (A, B, C) por separado.
- Contadores de la tabla de símbolos por fase (`compares`, y `moves` o `rotations`/`flips` según la implementación).
- Contadores de la cola de prioridad por fase (`less`, `exch`).
- Para los árboles: h, $\bar{d}$ y $\bar{d}_f$ al terminar la fase A.

Producir además las siguientes gráficas y análisis:

a) Crecimiento del vocabulario y distribución de frecuencias. Graficar M en función de N en escala log-log y ajustar $M = c\,N^{\beta}$, reportando c, β y R^2 (esta relación empírica se conoce como **ley de Heaps**). Usando el *top-K* con $K = 1000$ sobre el texto

completo, graficar en log-log la frecuencia en función de la posición en el *ranking* y estimar la pendiente (**ley de Zipf**). Estas dos relaciones se usan en el punto 7 para expresar los costos en función de N.

b) Escalamiento con N. Tabular y graficar el tiempo promedio de cada fase en función de N (escala log-log). Obtener la curva de mejor ajuste de la forma $T = a\,N^{b}$ para cada fase (regresión lineal sobre $\log T$ vs. $\log N$; en Excel/LibreOffice, línea de tendencia potencial mostrando ecuación y R^2), reportando b y R^2. Hacer lo mismo con los contadores de la fase A. Comparar los exponentes obtenidos con la predicción analítica del punto 7.

c) Forma de los árboles (todos los equipos, con `BST` y `RedBlackBST`). Para cada tamaño N, construir el índice con las dos implementaciones de árbol y graficar en función de $\lg M$: h y $\bar{d}$ de cada árbol, junto con las curvas de referencia $\lg M$, $2\ln M \approx 1.39\lg M$ (comparaciones esperadas de una búsqueda exitosa en un BST construido en orden aleatorio; la profundidad promedio es una menos) y $2\lg M$ (cota de altura del árbol rojo-negro). Graficar también $\bar{d}_f$. ¿Por qué $\bar{d}_f$ es mucho menor que $\bar{d}$ en ambos árboles? ¿Qué tiene que ver con el orden en que aparecen las palabras frecuentes en el texto?

d) Sensibilidad al orden de inserción. Tomar el vocabulario del texto completo con sus frecuencias e insertarlo con `put(w, f(w))` en una tabla vacía de la implementación asignada, partiendo de cuatro órdenes: (i) orden de **primera aparición** en el texto, (ii) orden **creciente** de llave, (iii) orden **decreciente** de llave y (iv) permutación **aleatoria** (semillas 1 a 10). En cada escenario medir el tiempo y los contadores de la construcción y, en los árboles, h, $\bar{d}$ y $\bar{d}_f$. Luego, sobre cada tabla construida, ejecutar como carga de búsqueda un `get` por cada una de las N palabras del texto completo y medir el tiempo y las comparaciones. Explicar los resultados en términos de la forma de la estructura. Observe que la implementación que es *mejor caso* en un escenario puede ser *peor caso* en otro: compárelo con lo que haría otra implementación de la tabla.

> **Advertencia sobre la recursión.** En el escenario (ii) un `BST` sin balancear degenera en una lista de altura $h = M - 1 \approx 23\,000$. Las implementaciones recursivas de `put`, `get` y `height` desbordarán la pila (en Python el límite por defecto es de unas 1000 llamadas; en Java y C# depende del tamaño de pila del hilo). El equipo debe decidir y documentar cómo lo resuelve: versiones iterativas de las operaciones, aumentar el límite de recursión o el tamaño de pila (por ejemplo, ejecutando la medición en un hilo con pila grande), o reducir M usando el vocabulario de un N menor. Si reduce M, reportar hasta dónde pudo llegar y por qué. **Estime primero** el tiempo con un caso pequeño y extrapole antes de lanzar el caso grande.

e) Efecto de K y construcción por *heapify* (todos los equipos). Sobre el índice del texto completo:

- Ejecutar la fase B con la cola de prioridad asignada para $K \in \{1, 10, 100, 1000, 10\,000\}$. Graficar `less`, `exch` y el tiempo en función de K (log-log) y comparar con la predicción del punto 7b. Si la cola asignada es cuadrática en K y el caso $K = 10\,000$ no termina en un tiempo razonable (del orden de un minuto), reportar hasta dónde llegó.
- Comparar, para los mismos valores de K, dos estrategias que usan el montículo binario: **(E1)** el esquema de la fase B (montículo de mínimos acotado a K) y **(E2)** construir un montículo de **máximos** con las M entradas mediante el constructor por *heapify* y ejecutar K veces `delMax`. Graficar `less` y tiempo de ambas en función de K e identificar el valor de K a partir del cual conviene cada una.

f) Comparación con una implementación de referencia. Únicamente para el texto completo y $K = 10$, repetir las tres fases reemplazando la combinación asignada por las estructuras de la biblioteca estándar del lenguaje:

- **Java:** `TreeMap<String, Integer>` (árbol rojo-negro; `subMap` para el rango del prefijo) y `PriorityQueue<Entrada>`.
- **C#:** `SortedDictionary<string, int>` con `StringComparer.Ordinal` y `PriorityQueue<TElement, TPriority>`. Como `SortedDictionary` no ofrece consultas por rango, usar para la fase C una lista ordenada de las llaves con `List<T>.BinarySearch`.
- **Python:** no hay tabla de símbolos ordenada en la biblioteca estándar. Usar `dict` para la fase A, `heapq.nlargest` para la fase B, y para la fase C una lista ordenada de las llaves con el módulo `bisect`.

Verificar que los resultados coinciden con los de su implementación (es una prueba de correctitud adicional) y cuantificar la diferencia de tiempo por fase y en total (factor o porcentaje).

7. Evaluación analítica

a) Tabla de símbolos (fase A). Para la implementación asignada, obtener la función tilde del número de comparaciones (y de `moves` en `BinarySearchST`) de la fase A **en función de N y M**, en el peor caso y en el caso típico. Indicar explícitamente el modelo de costo adoptado. Para el `BST`, el caso típico puede citar el resultado del texto guía para árboles construidos en orden aleatorio (búsqueda exitosa $\sim 2\ln M$ comparaciones) y debe discutir si el orden del texto se comporta como un orden aleatorio. Para el `RedBlackBST`, usar la cota de altura $2\lg M$. Para `BinarySearchST`, distinguir el costo de las búsquedas del costo de los

corrimientos: ¿cuál domina, y para qué valores de M? Luego **sustituir la ley de Heaps** medida en 6a ($M \approx c\,N^{\beta}$) para expresar el resultado solo en función de N. Incluir el proceso de deducción completo.

b) Cola de prioridad (fase B). Obtener la función tilde del número de `less` y `exch` de la fase B en función de M y K para la cola de prioridad asignada, en el peor y el mejor caso. Observe que la fase B ejecuta M inserciones y $M - K$ eliminaciones sobre una cola de tamaño a lo sumo $K + 1$. Justificar el costo de `insert` y `delMin` en la implementación asignada. Discutir: ¿alguna de las tres implementaciones hace exactamente el mismo número de comparaciones independientemente del orden de las entradas? Finalmente, analizar la optimización de comparar primero con `min()` y no insertar la entrada si no es más relevante que el mínimo: ¿cómo cambia el costo en el mejor y en el peor caso?

c) *Heapify* (todos los equipos). Demostrar que la construcción de un montículo de n elementos por *heapify* de abajo hacia arriba realiza menos de $2n$ comparaciones y menos de n intercambios (texto guía, proposición R). Con este resultado, obtener las funciones tilde de las estrategias E1 y E2 del punto 6e y determinar analíticamente el valor de K (en función de M) a partir del cual E2 es mejor que E1. Compararlo con el valor observado.

d) Autocompletado (fase C). Obtener el costo de una consulta de autocompletado en función de h (o de M), R y K', para la tabla de símbolos asignada. ¿Qué parte del costo depende de la estructura y qué parte depende solo de la respuesta? Usando que los rangos de los Q prefijos de longitud 2 son disjuntos y cubren todo el vocabulario salvo las palabras de una letra, obtener el costo total de la fase C. ¿Cuál de las tres tablas de símbolos es más eficiente para esta fase y por qué?

e) Costo total. Combinar (a), (b) y (d) en una expresión del costo total del programa en función de N (usando la ley de Heaps) e identificar **cuál fase domina asintóticamente**. ¿Cambia la fase dominante al elegir otra combinación de la tabla del punto 4?

f) Análisis de espacio. Estimar analíticamente la memoria requerida en función de M y K por: la tabla de símbolos asignada (arreglos paralelos frente a nodos con llave, valor, dos enlaces, `size` y, en el rojo-negro, el color) y la cola de prioridad. Suponer tamaños explícitos (por ejemplo, `int` = 4 B, referencia = 8 B, más el sobrecosto de objeto y de las cadenas según el lenguaje; documentar los supuestos) o medir con `sys.getsizeof` / perfilador del lenguaje. ¿Cuántos bytes por llave ocupa cada tabla de símbolos? ¿Qué fracción corresponde a las propias cadenas?

8. Discusión y reflexión

Responder de forma argumentada, apoyándose en los datos obtenidos:

- En el punto 6c, $\bar{d}_f$ resultó mucho menor que $\bar{d}$. Explique por qué el orden en que aparecen las palabras en un texto natural favorece a un BST sin balancear, y por qué este efecto **no** está garantizado: construya (o describa) un texto de la misma longitud y vocabulario para el cual el BST sin balancear sea muy costoso.
- Si el árbol rojo-negro garantiza altura logarítmica en cualquier escenario, ¿por qué un BST sin balancear podría ser una elección razonable para este problema? ¿Y por qué no lo sería para un índice que recibe las palabras de un diccionario ya ordenado?
- `BinarySearchST` tiene inserción de costo lineal en M. Con los datos de 6a y 6b, ¿es realmente una mala elección para este texto? ¿Qué propiedad del lenguaje natural la hace viable? ¿Hasta qué tamaño de texto lo seguiría siendo, según su ajuste?
- La tabla comparativa de las notas de clase indica que un árbol balanceado también puede implementar una cola de prioridad con todas las operaciones en $O(\log n)$. ¿Por qué se prefiere el montículo binario para la fase B? ¿En qué variante del problema convendría usar un árbol balanceado como cola de prioridad?
- Suponga que el texto llega como un **flujo** y se quiere mantener el *top-K* actualizado en cada palabra, sin recalcularlo desde cero. ¿Qué operación adicional necesitaría la cola de prioridad (pista: `changeKey`) y qué estructura auxiliar haría falta para ejecutarla en $O(\log K)$? No es necesario implementarlo.
- ¿Cuáles de las tres consultas del problema requieren que la tabla de símbolos sea **ordenada**? Si solo se necesitaran las consultas 1 y 2, ¿qué otra estructura podría ser más eficiente (tema del siguiente capítulo)?
- Si tuviera que reportar una sola cifra para caracterizar el costo de su programa, independiente de la máquina en que se ejecute, ¿reportaría comparaciones de llaves, intercambios en la cola o alguna combinación? Justifique con los datos.

Uso de asistentes de IA (metodología del curso)

En este período se **permite** el uso de asistentes de IA en la parte de codificación, **siempre** que se siga la metodología de la **Guía para el uso efectivo de IA en el curso**. El principio rector: **la IA implementa; ustedes deciden, especifican y verifican.** Traten la salida de la IA como una *dependencia poco confiable* que debe validarse antes de usarse.

El ciclo de trabajo exigido para cada componente es:

1. **Entender y descomponer el problema (sin IA):** reescribirlo con sus palabras, dividirlo en subproblemas y plantear la hipótesis de solución (qué estructura y qué algoritmo, y por qué).
2. **Elaborar el PRD** (*Project Requirements Document*): entradas/salidas y sus tipos, **precondiciones**, **postcondiciones**, **invariantes**, restricciones de complejidad temporal/espacial y **criterios de aceptación** (casos de prueba, incluidos los de borde).
3. **Registrar el prompt** utilizado, textualmente.
4. **Validar críticamente** la solución: leer y entender cada línea, ejecutar los casos de aceptación y **casos de borde**, **deducir la complejidad del código** (no aceptar la que afirma la IA) y comprobar precondiciones, postcondiciones e invariantes.
5. **Documentar problemas y correcciones**, distinguiendo errores de la IA de ambigüedades del propio PRD.
6. **Reflexionar** sobre la propiedad del resultado (*prueba de propiedad*).

Advertencia específica de este taller

Este taller es un caso donde las debilidades conocidas de los asistentes de IA se manifiestan de forma directa, y por eso conviene tenerlas presentes:

- **El análisis analítico (punto 7) es trabajo propio.** Un asistente puede afirmar con toda confianza que "las operaciones de un BST son $O(\log n)$", omitiendo que eso solo vale si el árbol está balanceado (el punto 6d muestra lo contrario), o que "*heapify* es $O(n \log n)$". También es habitual que exprese el costo en función de un único "n" sin distinguir N, M y K. Las funciones tilde deben deducirse del código propio y verificarse contra los contadores medidos.
- **Los invariantes se rompen en silencio.** Un `BST` cuyo campo `size` no se actualiza en algún camino de `put` (por ejemplo, al actualizar una llave existente, o tras una rotación) sigue respondiendo bien a `get` y a `keys()`, pero `rank` y `select` dan resultados erróneos. Un árbol rojo-negro al que le falta una rotación sigue siendo un BST válido, pero pierde la garantía de altura. Por eso se exigen los verificadores `isBST()`, `isSizeConsistent()`, `isRankConsistent()`, `is23()` e `isBalanced()` en las pruebas.
- **El código generado suele ser recursivo y dependiente del lenguaje.** Las versiones recursivas típicas fallan con los árboles degenerados del punto 6d, y en C# la comparación de cadenas por defecto depende de la cultura, lo que cambia el orden de las llaves y rompe el autocompletado. También es frecuente que un asistente "resuelva" la cola de prioridad con `heapq`, `PriorityQueue` o `TreeMap`, lo cual está prohibido salvo en el punto 6f.
- **La instrumentación es fácil de romper de forma silenciosa.** Un contador mal ubicado (por ejemplo, contar dos veces una comparación de tres vías en Python, o no contar las comparaciones dentro de `floor` o de `keys(lo, hi)`) no rompe ninguna prueba funcional, pero invalida las gráficas. Verifiquen cada contador contra un valor calculado a mano en un caso pequeño, y comprueben que los contadores son idénticos entre repeticiones.
- **Verificar no es validar.** La IA es buena *verificando* (que el código haga lo que dice) pero débil *validando* (que resuelva el problema correcto). Que sus estructuras pasen las pruebas no garantiza que el índice obtenido sea el que el enunciado pide: usen los valores de verificación del enunciado.

Componentes sobre los que se exige la documentación

Aplicar el ciclo completo (PRD + prompt + validación + problemas + reflexión) **como mínimo** a los siguientes tres componentes:

1. La **tabla de símbolos asignada** al equipo (punto 4), incluyendo sus operaciones ordenadas, sus verificadores de invariantes y su instrumentación.
2. La **cola de prioridad asignada** al equipo, incluyendo su instrumentación de comparaciones (`less`) e intercambios (`exch`).
3. El **programa de construcción del índice, consultas y medición** (punto 5), con especial atención a la separación de fases y a la exclusión de la lectura, la tokenización y la E/S de los bloques cronometrados.

Además, incluir **al menos el contrato** (precondiciones, postcondiciones e invariantes) del tokenizador y del ADT `Entrada`.

Plantilla del PRD (usar para cada componente)

```
# Título del problema / componente

## Requerimientos
- Entradas: <descripción y tipos de datos>
- Salidas: <descripción y tipos de datos>

## Contrato
- Precondiciones: <qué debe cumplirse antes de llamar cada función>
```

```
- Postcondiciones: <qué garantiza cada función al terminar>
- Invariantes: <propiedades que la estructura de datos mantiene siempre>

## Restricciones de implementación
- Complejidad temporal esperada: <p. ej. get en O(h), delMin en O(log K)>
- Complejidad espacial esperada: <p. ej. O(M)>
- Buenas prácticas / patrones exigidos: <...>
- Prohibiciones: <p. ej. no usar TreeMap, heapq ni PriorityQueue>

## Criterios de aceptación (casos de prueba)
- Caso típico: <entrada → salida esperada>
- Casos de borde: <vacío, un elemento, duplicados, orden creciente, ...>
- Casos de error: <entradas inválidas y comportamiento esperado>
```

Estructura de la documentación por cada componente

```
# <Nombre del componente>

## 1. Entender y descomponer (sin IA)
<El problema con sus palabras, subproblemas e hipótesis de solución>

## 2. PRD
<El PRD del componente, según la plantilla anterior>

## 3. Prompt(s) utilizado(s)
<Texto exacto del/los prompt(s). Indicar herramienta y fecha. Si no se usó IA, indicarlo
aquí>

## 4. Registro de validación crítica
- Casos de aceptación ejecutados (resultado)
- Casos de borde probados (resultado)
- Verificación manual de los contadores en un caso pequeño
- Análisis de complejidad PROPIO, deducido del código (no el que afirma la IA)
- Verificación de pre/postcondiciones e invariantes

## 5. Problemas encontrados y correcciones
<Qué falló, si fue error de la IA o del PRD, y cómo se corrigió (manual o con nuevo prompt)>

## 6. Reflexión final (prueba de propiedad)
<3–5 líneas: qué se aprendió, qué se haría distinto, si se podría reimplementar sin ayuda>
```

> **Importante:** El PRD, el registro de validación y la reflexión son **obligatorios aunque no usen IA** (especificar antes de codificar es una habilidad central del curso). El registro de prompts aplica cuando sí se use IA. Entregar código asistido por IA sin esta documentación se considera **entrega incompleta** y afecta directamente la calificación (ver rúbrica).

Formato: Markdown. Un único archivo `desarrollo-ia.md`, o una carpeta `desarrollo-ia/` con un archivo por componente y un `desarrollo-ia/README.md` que enlace las partes.

Entregables

- Código fuente del tokenizador, del ADT `Entrada`, de las tres tablas de símbolos, de las tres colas de prioridad y del programa de construcción del índice, consultas y medición. El análisis detallado se hace solo sobre la combinación asignada, pero las seis implementaciones (tres tablas de símbolos y tres colas de prioridad) deben entregarse, más la versión de referencia del punto 6f.
- Pruebas unitarias con verificaciones `assert` para todos los componentes, incluyendo los casos de borde y los verificadores de invariantes indicados en los puntos 1, 2 y 3.
- Hoja de cálculo (Excel o LibreOffice) con los datos experimentales tabulados, organizados por tamaño de N y por escenario (tiempos por fase, contadores de la tabla de símbolos y de la cola de prioridad por fase, M, h, $\bar{d}$, $\bar{d}_f$, K, semillas), con las gráficas y las curvas de mejor ajuste (incluyendo la expresión ajustada y el valor R^2).

- Documento de análisis (Word o PDF) que incluya:
  - El hash SHA-256 del equipo y la combinación asignada.
  - Los valores de verificación obtenidos (N, M, *top-10*, autocompletado de `dul` y `san`) y la fecha de descarga del texto.
  - El desarrollo analítico completo del punto 7.
  - Las gráficas e interpretaciones del punto 6.
  - Las respuestas argumentadas del punto 8.
  - Cómo se resolvió el problema de la recursión en el punto 6d y hasta dónde se pudo llegar.
- Documento `desarrollo-ia.md` (o la carpeta `desarrollo-ia/`) con la documentación de ingeniería con IA descrita en la sección **Uso de asistentes de IA**.

Condiciones Generales

- **Grupos:** Máximo 3 personas.
- **Fecha de entrega:** 30/Oct/2026, 11:59 p. m. (hora de Colombia), por Aula Digital.
- **Entrega:** Archivo comprimido nombrado `Taller4-<Apellido1Nombre1><ID1>-<Apellido2Nombre2><ID2>...` (.zip, .rar, .7z o .tgz). Ejemplo con dos integrantes: `Taller4-GarciaAna0001234-PerezLuis0005678.zip`.
- **No** incluir el archivo del texto (`pg2000.txt`) en la entrega; indicar en el informe la ruta esperada y la fecha de descarga.
- Marcar todos los archivos fuente y documentos anexos con los nombres de los miembros del equipo.

Nota importante: No utilizar bibliotecas externas ni estructuras de la biblioteca estándar (`dict`, `TreeMap`, `SortedDictionary`, `heapq`, `PriorityQueue`, ordenamientos de biblioteca, etc.) para la lógica principal de las tablas de símbolos ni de las colas de prioridad. Se permite su uso únicamente para la lectura y tokenización del texto (expresiones regulares), para la generación de permutaciones aleatorias, para la medición de tiempos y para la implementación de referencia solicitada en el punto 6f. En caso de utilizar estructuras de las bibliotecas del texto (algs4.jar), no anexar la biblioteca ni código compilado.

Política para entregas tardías (hora de Colombia, entrega por Aula Digital): Para entregas posteriores al 30/Oct/2026 11:59 p. m. aplican estas políticas:

- hasta el 31/Oct/2026 11:59 p. m., se califican sobre 4.0.
- hasta el 01/Nov/2026 11:59 p. m., se califican sobre 3.0.
- posteriores a esta fecha no se reciben.

Rúbrica

Todas las filas tienen igual peso en la nota final. Cada fila se califica de 0 a 5.

Actividad	0 – Nada	1 – Muy deficiente	2 – Deficiente	3 – Aceptable	4 – Bueno	5 – Excelente
a) Tokenización, ADT `Entrada` y valores de verificación	No se entrega código.	Código incompleto o no compila.	La tokenización no reproduce N y M, o el criterio de relevancia es incorrecto o no determinista.	Reproduce N y M; pruebas unitarias mínimas.	Reproduce todos los valores de verificación, con desempate correcto y pruebas representativas.	Correcto, inmutable, bien documentado, con pruebas que cubren tildes, `ñ`, `ü`, puntuación y empates.
b) Tablas de símbolos ordenadas e instrumentación	No se entrega código.	Código incompleto o no compila.	Solo una implementación funciona correctamente, o faltan las operaciones ordenadas.	Las tres funcionan; sin instrumentación, sin verificadores de invariantes o sin pruebas.	Las tres correctas con todas las operaciones ordenadas, instrumentadas (`compares` de tres vías contado una vez) y con	Las tres correctas e intercambiables, con verificadores de invariantes (`isBST`, `size`, `rank`, `is23`, `isBalanced`), pruebas de borde de las operaciones ordenadas, verificación

Actividad	0 – Nada	1 – Muy deficiente	2 – Deficiente	3 – Aceptable	4 – Bueno	5 – Excelente
					pruebas unitarias.	manual de los contadores y equivalencia de `keys()` entre implementaciones.
c) Colas de prioridad e instrumentación	No se entrega código.	Código incompleto o no compila.	Solo una implementación funciona correctamente.	Las tres funcionan; contadores o pruebas incompletos.	Las tres correctas, genéricas, instrumentadas (`less`/`exch`) y probadas; el montículo incluye *heapify*.	Las tres correctas, genéricas y probadas en casos de borde, con *heapify* lineal verificado, `isMinHeap` y verificación manual de los contadores.
d) Evaluación analítica (puntos 7a–7f)	Nada entregado.	Explicación irrelevante o equivocada.	Intento superficial con errores graves; confunde N, M y K, o supone altura logarítmica sin justificarla.	Análisis correcto de la tabla de símbolos, pero incompleto en la cola de prioridad, en *heapify* o en el espacio.	Análisis correcto de todas las fases y del espacio, distinguiendo claramente N, M y K y usando la ley de Heaps.	Análisis completo y riguroso, incluyendo la demostración de *heapify*, el punto de cruce entre E1 y E2, la fase dominante y su contraste con las mediciones.
e) Evaluación experimental (punto 6)	Nada entregado.	Código experimental no funcional.	Pocos tamaños o escenarios; fases no separadas en la medición.	Todos los escenarios, pero con pocas repeticiones, sin desviación estándar o sin curvas de ajuste.	Escenarios bien evaluados, fases separadas, contadores constantes entre repeticiones, ajustes con R^2 reportado.	Evaluación rigurosa: incluye leyes de Heaps y Zipf, forma de los árboles, sensibilidad al orden de inserción (resolviendo el problema de la recursión), efecto de K, E1 frente a E2 y comparación con la referencia.
f) Ingeniería con IA (PRD, validación crítica y reflexión)	No se entrega `desarrollo-ia.md`.	Se evidencia código pegado sin especificación ni verificación (*AI slop*), o no se documenta el uso de IA.	PRDs incompletos o genéricos; validación mínima sin casos de borde; no se deduce la complejidad por cuenta propia.	PRDs presentes con vacíos menores; validación presente pero superficial o aparentemente copiada de la IA.	PRDs completos para los tres componentes, prompts registrados textualmente, validación con casos de borde y complejidad deducida del código.	Documentación completa y genuina: contratos, invariantes verificados, verificación manual de los contadores, análisis de complejidad propio, problemas y correcciones documentados y

Actividad	0 – Nada	1 – Muy deficiente	2 – Deficiente	3 – Aceptable	4 – Bueno	5 – Excelente
						reflexión de propiedad honesta.
g) Discusión y reflexión (punto 8)	Nada entregado.	Respuestas ausentes o irrelevantes.	Respuestas superficiales, sin apoyo en los datos.	Responde de forma general la mayoría de las preguntas.	Respuestas fundamentadas en los resultados experimentales.	Discusión crítica y bien argumentada; contrasta análisis y experimentación y explica correctamente por qué el orden del texto favorece al BST sin balancear.